

MedSkills Catalyst

Technical Overview & Security Review Brief

Prepared for: External technical / security reviewer · **Date:** 7 August 2026

Prepared by: MedSkills Catalyst engineering

Scope: Public website, enrollment + payment flow, internal CRM/admin, and supporting infrastructure.

Purpose. A friend of our founder (Gagan Victor), a software engineer, has kindly offered to audit our site for security concerns and to sanity-check our payment integration. This is an honest, self-authored map of *what we run and how it's put together* so the review can start from facts rather than guesswork. Where we already know something is a simplification or a weak point, we've said so explicitly (see §9). If you only read one section, read §4 (Payment flow) and §9 (What to scrutinise).

1. The short version

We are **not** a single monolithic app. The site is two things living under one domain:

1. **A static marketing website** — plain **HTML / CSS / JavaScript** (landing pages, policy pages, brochures). No server logic. Fast, cheap, hard to attack.
2. **A dynamic application** — a **Next.js 15 / React 19 / TypeScript** app that handles the parts that need a server and a database: online **enrollment + payment**, an internal **CRM / admin**, a **careers** portal, and **webinar** registration.

Everything is hosted on **Vercel** (serverless). The database is **PostgreSQL on Supabase**. Payments run through **Razorpay**. Transactional email goes through **Resend**.

2. Technology stack (verified against the codebase)

Layer	Technology	Notes
Static site	HTML5, CSS, vanilla JS	~14 pages at repo root, mirrored into <code>public/</code> . Served as static files.
App framework	Next.js 15.3 (App Router)	React Server Components + Server Actions + API routes.
UI library	React 19	
Language	TypeScript 5.7	Strict typing across the app code.
Styling	Tailwind CSS 3.4, motion 12	Utility CSS + animation.
Forms & validation	react-hook-form 7, Zod 4	All user input is schema-validated with Zod.
ORM / DB access	Prisma 7 with @prisma/adapters-pg	Type-safe, parameterised queries (no hand-written SQL concatenation in app paths).
Database	PostgreSQL (Supabase)	Accessed via the Supabase transaction pooler.
File storage	Supabase Storage	Enrollment document uploads (ID, etc.).
Payments	Razorpay Standard Checkout	Server-side orders + signature verification + webhook.
Email	Resend (HTTP API)	Transactional (enrollment confirmations, invoices).
Analytics / marketing	PostHog, Google Analytics 4, Meta Pixel + Meta Conversions API	Meta CAPI is server-side.
Hosting / runtime	Vercel serverless (Node.js runtime)	
Auth (installed vs used)	@clerk/nextjs is installed but not wired in. Actual admin gate is a shared passcode → HMAC cookie (§5).	Flagged deliberately — see §9.

Node dependencies are pinned in `package.json` / `package-lock.json`. No custom C/native modules.

3. Repository & application structure

/ (repo root)	
├─ *.html	Static marketing site (index, about, policies, brochures...)
├─ public/	Mirror of the static pages served by Vercel
├─ assets/	Shared static JS/CSS/images (incl. meta-pixel.js)
├─ src/	
│ └─ app/	Next.js App Router
│ └─ foundation/	Foundation course landing + enroll
│ └─ advanced/	Advanced module landing + enroll
│ └─ enroll/[token]/	Token-based (personalised) enrollment + payment
│ └─ webinar/	Webinar registration + calendar.ics
│ └─ careers/	Public job listings + application form
│ └─ admin/	Internal CRM (leads, careers) – passcode-gated
│ └─ api/	Server endpoints (see §4/§6)
│ └─ enroll/upload/	Document upload → Supabase Storage
│ └─ webhooks/razorpay/	Payment webhook (authoritative)
│ └─ invoices/	GST invoice generation
│ └─ careers/	Job application + admin
│ └─ qa-register/ register/	Webinar/lead capture
│ └─ admin/leads/export/	CSV/XLSX lead export (gated)
│ └─ lib/	Cross-cutting server helpers
│ └─ auth.ts	Passcode session + role check
│ └─ db.ts	Prisma client (singleton)
│ └─ razorpay.ts	Order creation + signature verification
│ └─ invoices.ts	GST invoice builder
│ └─ email.ts	Resend wrapper
│ └─ supabase.ts	Storage admin client
│ └─ meta-capi.ts	Server-side Meta Conversions API
│ └─ modules/	Domain logic (enroll, leads) – schemas + services
│ └─ generated/prisma/	Prisma client (generated, git-ignored)
├─ prisma/schema.prisma	Database schema (source of truth for tables)
├─ supabase/*.sql	Raw SQL schema snapshots / migrations
├─ docs/CRM_ARCHITECTURE.md	CRM design doc
├─ next.config.ts	URL rewrites (static pages → clean URLs)
├─ vercel.json	Hosting config / headers
└─ .env.example	Documents every env var (values blank – see §7)

Two-surface design in one line: `next.config.ts` rewrites clean URLs (e.g. `/`, `/refund-policy`) to the static HTML files, while the dynamic routes under `src/app/**` are the real application. This is intentional — the marketing pages carry no server logic and therefore no server-side attack surface.

4. Payment flow (Razorpay)

This is the highest-value area to review. The design principle throughout is **never trust the browser for money**.

Order creation (server-side). When a user enrolls, the **amount is computed on the server from the database** (`EnrollmentLink.price_paise` / course price), never read from the client. The server calls Razorpay's Orders API with that trusted amount. Code: `src/lib/razorpay.ts` → `createRazorpayOrder()`, called from `src/modules/enroll/service.ts`. Amount is validated (integer, ≥ 100 paise).

Checkout. The browser runs Razorpay's hosted `checkout.js`. Only the **publishable** key id (`NEXT_PUBLIC_RAZORPAY_KEY_ID`) ever reaches the browser. The **key secret never leaves the server** (`src/lib/razorpay.ts` is marked `import "server-only"`).

Two independent verification paths:

1. **Client callback signature** — after checkout we verify `HMAC_SHA256(order_id|payment_id, KEY_SECRET) === razorpay_signature`, using a **constant-time** hex comparison (`crypto.timingSafeEqual`).
2. **Webhook (authoritative)** — `POST /api/webhooks/razorpay` is treated as the source of truth for "money actually moved." It:
 - reads the **raw request body** (no framework parsing) and verifies `X-Razorpay-Signature` against `RAZORPAY_WEBHOOK_SECRET`;
 - **deduplicates** on the `x-razorpay-event-id` header via a unique DB constraint (`WebhookEvent`);
 - on `payment.captured` / `order.paid`, runs **idempotent fulfillment** (record capture → generate invoice → send email);
 - returns **non-200 to force Razorpay to retry** if any fulfillment step didn't complete, and marks the event processed only once it did.

Why this matters for the audit: fulfillment is safe to repeat, duplicate deliveries are no-ops, and a forged / failed-signature webhook is rejected with `400` before any DB write. We'd welcome a second pair of eyes on the retry / idempotency edges specifically.

Keys. We can run on Razorpay **test** keys and swap to **live** keys via environment variables only — no code change. Confirming the live-key configuration is one thing we'd like verified.

5. Authentication & access control

There are two distinct trust zones:

- **Public** — marketing pages, course landing pages, enrollment forms, webinar/careers forms. No login; protected by input validation and per-record gating (e.g. uploads require a valid live enrollment token).
- **Internal (CRM/admin)** — `src/app/admin/**` and the sensitive API routes (lead export, invoices, careers admin).

How admin is gated today (please read carefully): the internal area uses a **single shared passcode** (`ADMIN_PASSCODE`, an env var). On success we set an **httpOnly cookie** whose value is `HMAC_SHA256("msc-admin-v1", passcode)` — so the cookie can't be forged without the secret. Passcode and cookie checks use **constant-time comparison** (`timingSafeEqual`). Enforcement is per-route/page via `requireStaff()` / `getStaff()` in `src/lib/auth.ts`; a role hierarchy (`ADMIN > ACCOUNTS > COUNSELOR > VIEWER`) exists in the schema, but in practice every passcode session maps to a single ADMIN staff row. Two older API endpoints (`/api/careers/admin`, `/api/invoices`) check the passcode directly from an `x-admin-passcode` header rather than the cookie session.

Deliberate flag. This shared-passcode scheme is intentionally simple and is the **main thing we want your opinion on** (see §9). `@clerk/nextjs` is already a dependency in anticipation of moving to real per-user accounts, but it is **not yet wired in**. There is also **no `middleware.ts`** — access control is enforced inside each protected page/route, not by a global gate, so it's worth confirming no protected route is missing its check.

6. Data model & storage

- **Database:** PostgreSQL (Supabase), schema in `prisma/schema.prisma` (leads, staff users, courses, batches, enrollment links, enrollments, payments, invoices, webhook events, activities, careers applications). Raw SQL snapshots in `supabase/*.sql`.
- **DB access:** Prisma with the `pg` adapter, through the Supabase transaction pooler. Queries are parameterised by the ORM.
- **File uploads:** enrollment documents go to a **Supabase Storage** bucket via a server-only admin client (`src/lib/supabase.ts`). The upload endpoint (`/api/enroll/upload`) gates writes to a **live enrollment link** and enforces an allow-list of MIME types / extensions, so the bucket can't be used as open/anonymous storage.
- **PII handled:** names, emails, phone numbers, uploaded identity documents, payment/invoice records. This is the data most worth threat-modelling.

7. Secrets & configuration

- All secrets are **environment variables**, injected on Vercel — **none are committed**. `.gitignore` excludes `.env`, `.env.local`, and `.env*`; the only tracked env file is `.env.example`, which lists variable *names* with blank/placeholder values and inline docs. We confirmed no real `.env` file is tracked in git.
- Client-exposed values are namespaced `NEXT_PUBLIC_*` (pixel id, GA id, Razorpay publishable key) — intentionally public identifiers, not secrets.
- Server-only secrets: `RAZORPAY_KEY_SECRET`, `RAZORPAY_WEBHOOK_SECRET`, `SUPABASE_SERVICE_ROLE_KEY`, `RESEND_API_KEY`, `META_CAPI_ACCESS_TOKEN`, `ADMIN_PASSCODE`, `DATABASE_URL`. Files touching these are marked `import "server-only"` to prevent accidental client bundling.

8. Third-party services (external trust surface)

Service	Used for	Secret held
Vercel	Hosting, serverless functions, env management	Platform account
Supabase	PostgreSQL + object storage	SUPABASE_SERVICE_ROLE_KEY , DATABASE_URL
Razorpay	Payment processing	RAZORPAY_KEY_SECRET , RAZORPAY_WEBHOOK_SECRET
Resend	Transactional email	RESEND_API_KEY
Meta (Facebook)	Pixel + Conversions API (marketing)	META_CAPI_ACCESS_TOKEN
Google Analytics / PostHog	Web analytics	Public measurement IDs

9. Things we'd specifically like you to scrutinise

We're being candid here so your time goes to what matters:

1. **Admin authentication.** The shared-passcode → HMAC-cookie scheme (§5). Is it adequate for an internal CRM holding customer PII, or should we finish migrating to per-user accounts (Clerk is already installed)? Any session-fixation / cookie-scope / logout concerns?
2. **Payment integrity.** The end-to-end Razorpay flow (§4) — order/amount trust boundary, signature verification, and especially the **webhook idempotency + retry** logic. Can a payment be double-fulfilled or fulfilled without real capture?
3. **Route-level access control.** Because there's no global middleware, is any protected page or API route missing its `requireStaff` / passcode check? The lead-export and invoice endpoints are the most sensitive.
4. **File upload endpoint.** `/api/enroll/upload` — MIME/extension allow-listing, size limits, token gating, and whether the storage bucket is appropriately private.
5. **PII exposure.** Anywhere identity documents, emails, or phone numbers could leak (storage bucket ACLs, export endpoints, error messages/logs).
6. **Dependency & config hygiene.** Anything in `package-lock.json` , `next.config.ts` , `vercel.json` , or security headers (CSP, HSTS, etc.) you'd tighten.

10. How to get oriented quickly

- **Run locally:** `npm install` → set env vars from `.env.example` → `npm run dev` .
- **Start reading here:** `src/lib/razorpay.ts` , `src/app/api/webhooks/razorpay/route.ts` , `src/lib/auth.ts` , `src/app/api/enroll/upload/route.ts` , `prisma/schema.prisma` .
- **Design docs:** `docs/CRM_ARCHITECTURE.md` .

Happy to walk through any part live, grant read access to the repo and Vercel/Supabase dashboards as needed, and answer follow-ups.

